

Project Proposal

FairMatch AI

An AI-Powered System That Matches Candidates to Internships/Jobs Fairly and Automatically

Submitted by

Sejal Somkuwar

Master of Computer Applications (MCA)

Submitted to

Dr. Tanmoy Kanti Das

National Institute of Technology, Raipur

1. Problem Statement - What is the Problem?

Every year, thousands of students apply for internships and jobs, and companies or government schemes have only a limited number of seats to offer. Right now, most of this matching is done manually, or using simple keyword search (for example, just checking if the word Python appears in a resume).

This causes real problems:

- It takes a very long time to go through thousands of applications by hand.
- Different reviewers judge the same resume differently, so results are inconsistent.
- Keyword matching is not smart - it cannot tell that someone who knows machine learning is also a good fit for a role asking for AI skills, because the exact words are different.
- Some good candidates may never get matched properly, simply because of how the sorting was done, which is unfair.

In short: we need a system that actually understands what a candidate is good at, compares it properly with what a role needs, and then assigns people to opportunities in a way that is fast, fair, and easy to explain.

2. Proposed Solution

FairMatch AI is a system that automatically reads candidate resumes and internship/job postings, understands their meaning using AI, and then matches candidates to the best-fitting opportunities - while making sure every seat is filled fairly and every decision can be explained.

In simple terms, the system does four things:

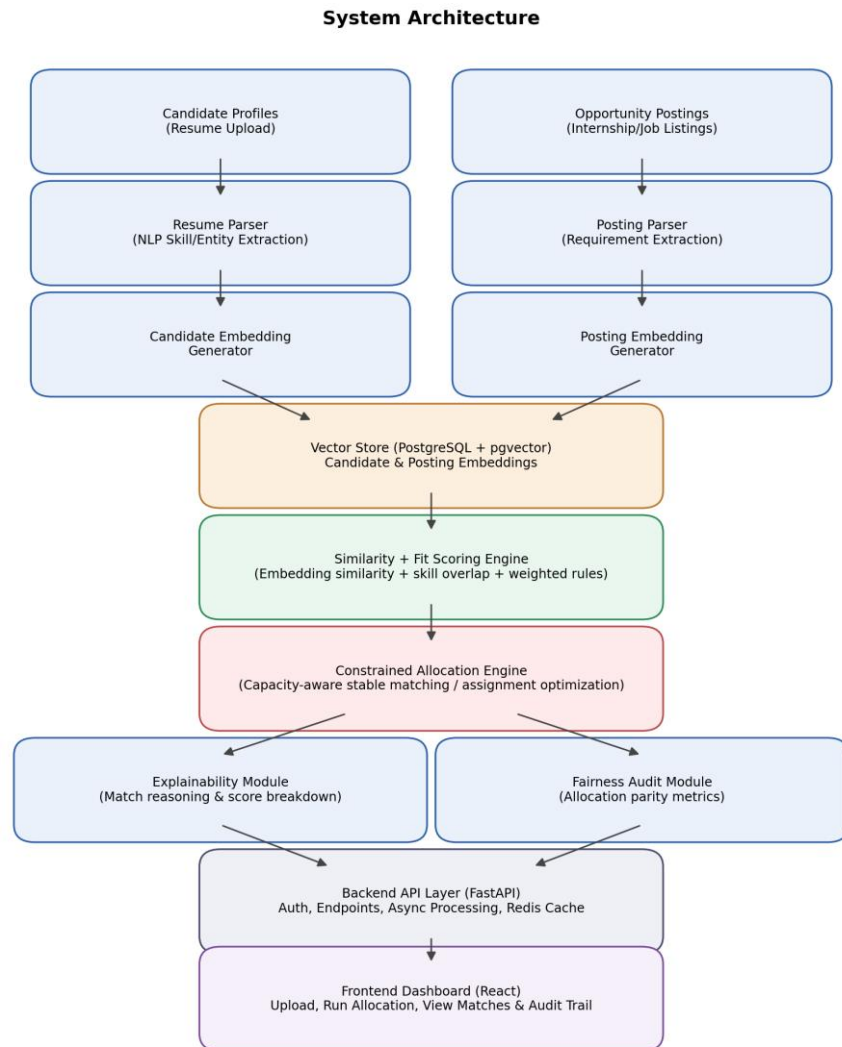
- **Reads and understands** resumes and job posts (not just keywords, but actual meaning) using AI.
- **Scores how good a match is** between each candidate and each opportunity.
- **Assigns candidates to opportunities** in a smart way, respecting the fact that each opportunity has a limited number of seats.
- **Explains every decision** - so we know exactly why a candidate was matched to a particular opportunity, and checks that the results are fair across different groups of candidates.

3. Objectives

- Automatically extract skills and requirements from resumes and job/internship postings using NLP (Natural Language Processing).
- Use AI (embeddings) to understand the real meaning of skills, not just match exact keywords.
- Calculate a fit score between every candidate and every opportunity.
- Design an allocation method that assigns candidates to opportunities while respecting seat limits, instead of just recommending options with no real assignment.
- Make every match explainable - show which skills or factors led to a match.
- Check whether the final allocation is fair across candidate groups.
- Build a working demo: a simple website where you can upload resumes/postings and see the matching results.

4. System Architecture

The diagram below shows how data moves through the system, step by step, from raw resumes and job posts at the top, to the final results shown on the dashboard at the bottom.



How to read this diagram (in simple words):

- **Step 1 - Input:** Candidates upload their resumes, and companies/organizations upload their opportunity postings.
- **Step 2 - Parsing:** The system reads the text and pulls out useful information - skills, qualifications, and requirements.
- **Step 3 - Embeddings:** Each resume and posting is converted into a set of numbers (called an embedding) that captures its real meaning, so the computer can compare them mathematically.
- **Step 4 - Scoring:** The system compares every candidate with every opportunity and gives a fit score, based on how similar their meanings are and how many skills overlap.
- **Step 5 - Allocation:** Instead of just picking the top score for everyone (which could fill one opportunity with too many people and leave another empty), the system runs an algorithm that assigns candidates to opportunities while respecting the number of seats available.
- **Step 6 - Explainability and Fairness:** For each match, the system shows why it was made, and checks whether the overall results are fair across different candidate groups.

- **Step 7 - Output:** Everything is shown through a website (dashboard) where results can be viewed and reviewed.

5. Methodology - How It Will Be Built

5.1 Reading Resumes and Postings

We will use an NLP library called **spaCy** to read resumes and job postings and pull out important details like skills, education, and experience automatically.

5.2 Understanding Meaning (Embeddings)

We will convert each resume and posting into a set of numbers using a pre-trained AI model (called a Sentence-Transformer). This lets the computer understand that machine learning and AI are related, even though they are different words.

5.3 Calculating Fit Score

For every candidate-opportunity pair, we calculate a score based on: (a) how similar their meanings are (from the embeddings), and (b) how many exact skills overlap. Both are combined into one final score.

5.4 Assigning Candidates to Opportunities

This is the most important part. Instead of just recommending the top match (which can overload one popular opportunity), we will use a proper allocation algorithm that assigns people while respecting the number of seats each opportunity has. We will try two well-known approaches and compare them:

- **Hungarian Algorithm** - a method that finds the best possible overall assignment given the seat limits.
- **Gale-Shapley Algorithm** - a method that matches two sides (candidates and opportunities) so that no one can be made happier by swapping their match (this is called a stable match).

5.5 Explaining the Results

For every match made, the system will show the top reasons behind it - for example, matched because of overlapping skills in Python and Data Analysis, with 87% similarity. This way, the result is never a mystery.

5.6 Checking Fairness

After allocation is done, the system will check whether outcomes are balanced across different candidate groups, so no group is unfairly left out.

6. Module Breakdown

Module	What It Does
Profile Upload	Lets candidates upload resumes and organizations upload postings
Resume/Posting Reader (NLP)	Extracts skills, qualifications, and requirements from text
Embedding Generator	Converts text into AI-understandable numeric form
Scoring Engine	Calculates fit score between each candidate and opportunity
Allocation Engine	Assigns candidates to opportunities, respecting seat limits

Module	What It Does
Explainability Module	Shows the reason behind every match
Fairness Checker	Verifies that results are balanced across candidate groups
Backend (API)	Connects all modules together and handles requests securely
Dashboard (Frontend)	Website where users upload data and view match results

7. Technologies to Be Used

Part of the System	Technology	Why This is Used
Backend (server)	FastAPI (Python)	Fast and simple to build APIs with
Reading resumes/postings	spaCy (NLP)	Extracts skills and details from text
Understanding meaning	Sentence-Transformers	Converts text into meaningful number form
Database	PostgreSQL + pgvector	Stores data and allows fast similarity search
Speeding things up	Redis	Caches repeated calculations for speed
Allocation logic	SciPy (Hungarian Algorithm) / Custom Gale-Shapley code	Assigns candidates fairly under seat limits
Website (frontend)	React.js	Builds the dashboard users interact with
Login security	JWT	Keeps user data and accounts secure
Code management	Git / GitHub	Tracks changes and versions of the code
Running the project	Docker	Makes the project easy to run anywhere

8. Expected Outcome

- A working prototype that can take in resumes and postings, and produce a fair, explainable allocation of candidates to opportunities.
- A comparison of two allocation methods (Hungarian Algorithm vs. Gale-Shapley) showing which works better and why.
- A simple, functional dashboard showing the entire process from upload to final results.
- A clear, documented explanation of how the system works, so it can be understood, evaluated, and extended in the future.

9. Why This Project

This project combines two important skills together - **backend engineering** (APIs, databases, real system design) and **AI/NLP** (understanding text, embeddings, matching algorithms). It solves a real, relatable problem (matching people to opportunities fairly), and the logic, design, and code for this project are built independently as original work.